

TaskOrbit Conversational Agent

User Documentation

Sprint 13 — Final Release Consolidated Documentation

Generated: 2026-07-12

Source: GitHub Wiki (`amos2026ss04-taskorbit-conversational-agent.wiki`)

Built by: AMOS SS2026 Team 04, in collaboration with Meisterwerk App GmbH, Berlin

Table of Contents

1. [User Documentation](#)
 2. [Agent Task Schema](#)
 3. [Agent Task Example](#)
 4. [Agent Configuration Management](#)
 5. [Agent Persona Guardrails](#)
 6. [End Call Tool](#)
 7. [Generic External Tool Adapter](#)
 8. [API Reference](#)
-

User Documentation

TaskOrbit User Documentation

What is TaskOrbit

TaskOrbit is a conversational AI platform designed to handle multi-step tasks reliably over voice and chat. Traditional voice agents fail in production because their behaviour is governed by a single, monolithic system prompt that grows unmanageable as requirements expand — causing instruction drift, inconsistent responses, and slow iteration cycles. TaskOrbit replaces the single giant prompt with a structured, per-agent configuration (persona, tools, and workflow rules) plus a lightweight orchestration engine that drives the conversation turn by turn.

Key Concepts

Concept	Description
Agent Configuration	The full definition of one agent: persona, greeting, STT/LLM/TTS providers, tools, and workflow rules. See Agent Task Schema
Conversation	One conversation run, identified by <code>conversation_id</code> , holding message history, extracted slots, and workflow progress
Orchestration Engine	The backend component that evaluates intent, routes to an agent, calls tools, and drives the conversation turn by turn
Tool	A callable action an agent can trigger: <code>data_extraction</code> , <code>agent_transfer</code> , <code>end_call</code> , Or <code>external_api</code>
Confirmation	A pause point before a tool with <code>confirmation.required: true</code> actually executes, requiring the user to confirm or reject
Template / Custom Agent	A built-in default agent, or a user's customized copy of one (see the <code>/v1/user-agents</code> flow below)

Getting Started

Prerequisites

Docker Desktop is the only required local installation. Python, Poetry, Node.js, and PostgreSQL all run inside containers.

Running the Full Stack

1. Clone the repository.
2. Copy the environment template files:

```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env.local
```

3. Start the stack:

```
docker compose up --build
```

4. This starts **eight services**:
 - `postgres` — PostgreSQL, exposed on host port **5435** (mapped to 5432 in the container, to avoid clashing with a local PostgreSQL install)
 - `backend` — FastAPI HTTP API, port 8000
 - `worker` — LiveKit voice agent (STT → orchestrator → TTS), port 8001

- `frontend` — Vite dev server with hot reload, port 5173
- `prometheus` — metrics, port 9090
- `loki` — log aggregation, port 3100
- `promtail` — log shipper (no exposed port)
- `grafana` — dashboards, port 3000

5. Open <http://localhost:5173> in your browser.

The first run takes several minutes while Docker pulls and builds base images. Subsequent starts are fast.

Using the Agent

Example: A Complete Agent

A worked example bundled with this documentation is a **Booking Assistant** that collects restaurant reservation details — date, time, and party size — and confirms them before booking:

1. **Greet** — the agent sends its configured `greeting` immediately (no LLM call needed for this turn).
2. **Collect** — the agent gathers the reservation details via a `data_extraction` tool.
3. **Confirm** — because that tool requires confirmation, the conversation pauses and asks the user to confirm before proceeding.
4. **Book** — the user confirms and the turn completes.

This agent is defined as a plain JSON `AgentConfig` object sent directly in the API request — there is no task file on disk to point at. See [Agent Task Example](#) for the complete, working request/response pair, and [Agent Task Schema](#) for the full field reference. For interactive, always-current API docs, see <http://localhost:8000/docs>.

Agent Configuration

Agents can be built and saved through the configuration UI at <http://localhost:5173/config>, or managed directly via the API.

Two related persistence paths exist:

- `/v1/agent-configs` — flat save/load/update/delete of a named configuration. See [Agent Configuration Management](#) for the full request/response contract.
- `/v1/user-agents` — a template-aware flow, described next.

Saving and customizing an agent (`/v1/user-agents`)

- GET `/v1/user-agents` returns every agent visible to you: your own customized copies, plus every built-in template you haven't touched yet. The list never shrinks — untouched templates always show up alongside your customizations.
- Editing a template you haven't saved before and clicking **Save** clones that template into your own copy on first save (copy-on-write); the shared template itself is never modified. Saving again just updates your copy.
- **Save as new** always creates an entirely new, independent agent — it never overwrites whatever you currently have loaded.
- Deleting your customized copy doesn't delete the template — the agent simply reverts to showing the original, unmodified template.

Voice Interface

1. Open <http://localhost:5173>.
2. Click **Start Session** — the browser will request microphone access.
3. The agent delivers its greeting; the microphone is automatically enabled after the greeting plays.
4. Speak naturally. A turn commits automatically after approximately two seconds of silence. You can also click **Send** to commit the turn immediately.
5. The agent's response is synthesized via ElevenLabs and plays through your browser speakers.
6. To interrupt the agent mid-response, start speaking — barge-in detection cuts the agent's audio immediately and begins processing your new turn.
7. Click **End Call** to terminate the session.

Persona Guardrails

Each agent can define an explicit operating scope via `persona_constraints` in its configuration: `scope` (a description of what the agent should stay within), `out_of_scope` (a list of categories to refuse — e.g. medical or legal advice), and `refusal_template` (the

message returned when a request falls outside scope). The orchestration engine enforces this at the system-prompt level automatically. See [Agent Task Schema](#) for the exact fields.

API Access

TaskOrbit exposes a REST API for programmatic use. The base URL when running locally is `http://localhost:8000`. All routes are prefixed with `/v1` except `/health`. Interactive documentation is available at <http://localhost:8000/docs>.

Method	Path	Purpose
GET	<code>/health</code>	Service health check
POST	<code>/v1/conversations/process</code>	Submit a conversation turn and receive the agent's reply
POST	<code>/v1/conversations/stream</code>	Same, streamed as Server-Sent Events
GET	<code>/v1/conversations</code>	List conversations
GET	<code>/v1/conversations/{id}/history</code>	Full history for one conversation
GET	<code>/v1/agent-configs</code>	List all saved agent configurations
GET	<code>/v1/agent-configs/{id}</code>	Load a specific saved configuration
POST	<code>/v1/agent-configs</code>	Save a new agent configuration
GET	<code>/v1/user-agents</code>	List agents visible to you (templates + your customizations)
POST	<code>/v1/user-agents</code>	Save a brand-new agent
PUT	<code>/v1/user-agents/{id}</code>	Customize an agent (update or clone-from-template)
POST	<code>/v1/livekit/token</code>	Mint a LiveKit room token for a voice session
POST	<code>/v1/tts/synthesize</code>	Text-to-speech fallback for typed input

This is a user-facing summary, not the complete contract. For every endpoint, full request/response field tables, and error codes, see [API Reference](#) and [Agent Configuration Management](#). For system design, see [System Architecture](#).

See also: [Agent Task Schema](#) · [Agent Task Example](#) · [API Reference](#) · [Agent Configuration Management](#) · [System Architecture](#)

Agent Task Schema

Agent Task Schema

This page documents the `AgentConfig` Pydantic schema — the strict, validated shape the orchestration engine expects on `ConversationRequest.agent_config` for `POST /v1/conversations/process` and `POST /v1/conversations/stream`.

Source of truth: `backend/src/taskorbit/types.py` (class `AgentConfig` and its sub-models). This module is the single source of truth for every data structure exchanged between the API layer, orchestration engine, agents, and tools, and it mirrors the `AgentConfig` TypeScript type in `frontend/src/types/agentConfig.ts`.

This is not the same object as the opaque `config blob` stored by `POST /v1/agent-configs` or `POST /v1/user-agents`. Those two endpoints persist whatever JSON the frontend sends as `dict[str, Any]`, with no schema enforcement — see [Agent Configuration Management](#). The frontend adapts that wider shape down to the strict `AgentConfig` below (via `conversationApi.ts`) before calling the conversation endpoints. This page describes only the strict, enforced shape.

There used to be a separate JSON *task file* format (`agent_task.schema.json`, with a `task` block and `agent` block, loaded from disk via `task_path`). **That format and the `/pipeline/run` endpoint that consumed it are gone.** Agent definitions are now plain `AgentConfig` objects sent inline in the request body — there is no file to point at.

Top-level fields

```
class AgentConfig(BaseModel):
    id: str
    name: str
    persona: str
    greeting: str
    stt: STTConfig
    llm: LLMConfig
    tts: TTSTConfig
    tools: list[ToolDefinition] = []
    persona_constraints: PersonaConstraints | None = None
    context_limit: ContextLimitConfig | None = None
    workflow_dependencies: list[str] = []
    workflow_rules: list[WorkflowRule] = []
    allowed_handoffs: list[str] = []
```

Field	Type	Required	Description
<code>id</code>	string	✓	Unique agent identifier
<code>name</code>	string	✓	Display name
<code>persona</code>	string	✓	System prompt / persona description sent to the LLM
<code>greeting</code>	string	✓	Opening message. Persisted as the first assistant message when a conversation is auto-created
<code>stt</code>	STTConfig	✓ (has defaults)	Speech-to-text provider config
<code>llm</code>	LLMConfig	✓ (has defaults)	LLM provider config

tts	TTSConfig	✅ (has defaults)	Text-to-speech provider config
tools	array of ToolDefinition	—	Tools this agent can invoke
persona_constraints	PersonaConstraints Or null	—	Optional scope/refusal guardrails
context_limit	ContextLimitConfig Or null	—	Conversation history truncation policy
workflow_dependencies	array of string	—	Agent ids (workflow blocks) that must complete unconditionally
workflow_rules	array of WorkflowRule	—	Conditional branches evaluated after intent routing; overrides workflow_dependencies for the matched turn when non-empty
allowed_handoffs	array of string	—	Agent ids this agent is permitted to transfer to

⚠️ **Superseded field names — do not use**

Older documentation and examples in this wiki used a different, now-incorrect field set. None of these exist on `AgentConfig` :

Old (wrong) name	Current field
agent_id	id
instructions	persona
first_message	greeting
variables	<i>(removed — no equivalent top-level field)</i>
engine	<i>(does not exist)</i>
vad	<i>(does not exist on AgentConfig; VAD wiring lives only in the LiveKit voice worker runtime, not the agent schema)</i>
confirmations (agent-level list)	confirmation is per-tool: ToolDefinition.confirmation (singular)
language (agent-level)	nested under stt.language

Sub-schemas

STTConfig

```
class STTConfig(BaseModel):
    provider: STTProvider = STTProvider.DEEPGRAM # "deepgram" | "elevenlabs"
    language: str = "multi"
    model: str = "nova-3"
```

LLMConfig

```
class LLMConfig(BaseModel):
    provider: LLMProvider = LLMProvider.OPENAI # "openai" | "google" | "openrouter" | "ollama"
```

```
model: str = "gpt-4o-mini"
```

TTSTConfig

```
class TTSTConfig(BaseModel):
    provider: TTSPProvider = TTSPProvider.ELEVENLABS # "elevenlabs" | "deepgram"
    voice_id: str = "21m00Tcm4TlvDq8ikWAM"
    model: str = "eleven_multilingual_v2"
```

PersonaConstraints

Optional operating-scope guardrail. Augments the system prompt so the LLM stays in role on off-topic input; a constraints object with no fields populated is a no-op.

```
class PersonaConstraints(BaseModel):
    scope: str | None = None
    out_of_scope: list[str] = []
    refusal_template: str | None = None
```

ContextLimitConfig

Controls how many messages the agent remembers before the oldest are FIFO-truncated. The system prompt is always protected and never truncated. Only `message_count` truncation is currently supported.

```
class ContextLimitConfig(BaseModel):
    type: Literal["message_count"] = "message_count"
    value: int = 50 # 2-499
```

WorkflowRule

One conditional branch, evaluated after intent routing.

```
class WorkflowRuleWhen(BaseModel):
    intent: str | None = None # match when detected intent equals this
    agent_name: str | None = None # match when routed agent_name equals this
    else_branch: bool = False # JSON key: "else" – matches if no prior rule matched

class WorkflowRule(BaseModel):
    when: WorkflowRuleWhen
    dependencies: list[str] = [] # agent ids (workflow blocks) to run when matched
```

Validation rule enforced on `AgentConfig`: at most one rule may set `"else": true`, and if present it must be the **last** entry in `workflow_rules`.

ToolDefinition

```
class ConfirmationConfig(BaseModel):
    required: bool = False
    prompt: str = ""

class ToolDefinition(BaseModel):
    id: str
    name: str
```

```
type: ToolType          # "data_extraction" | "agent_transfer" | "end_call" | "external_api"
description: str = ""
confirmation: ConfirmationConfig = ConfirmationConfig()
parameters: dict[str, Any] = {}
```

`confirmation.required = true` causes the conversation to pause with `status: "confirmation_required"` (see [API Reference](#)) before the tool actually executes; the caller answers via `ConversationRequest.confirmation_id + decision`.

The shape of `parameters` depends on `type`:

data_extraction

`parameters` is a flat dict of slot name → value. A tool call is considered complete once every value is non- `null`.

```
{ "date": null, "time": null, "party_size": null }
```

end_call

No parameters required — `parameters` may be `{}`. Signals that the conversation should end; the frontend/worker performs the actual session teardown after seeing `status: "ended"`.

agent_transfer

```
{ "target_agent_id": "technical_support", "conversation_history": [] }
```

Field	Type	Description
target_agent_id	string	A built-in agent name, or the UUID hex id of a saved custom agent configuration
conversation_history	array of message objects	Full message list passed through so the receiving agent has complete context

external_api

The richest tool type — its entire behavior is driven by configuration rather than code, so new external integrations (CRM lookups, weather APIs, outbound webhooks) can be added without a code change.

```
{
  "request": {
    "method": "GET",
    "url": "https://api.example.com/orders/{{args.order_id}}",
    "headers": { "Authorization": "Bearer {{env.EXAMPLE_API_KEY}}" },
    "query": {},
    "body": null,
    "timeout_seconds": 10
  },
  "response": {
    "extract": { "status": "order.status" },
    "success_when": { "status_in": [200, 201] }
  },
  "auth": { "allowed_env": ["EXAMPLE_API_KEY"] },
  "error_mapping": { "HTTP_4XX": "That order number wasn't found." },
  "args_schema": {
    "required": ["order_id"],
    "properties": { "order_id": { "type": "string" } }
  }
}
```

```
  },  
  "args": { "order_id": "12345" }  
}
```

Key behaviors:

- `{{env.X}}` and `{{args.Y}}` templating is substituted into `url`, `headers`, `query`, and `body`. `env` references are gated by `auth.allowed_env` — referencing an env var not on that whitelist fails the call rather than silently leaking it.
- `response.extract` maps output field name → dot-path into the upstream JSON response, so the LLM always sees a stable shape regardless of the vendor's payload.
- Failures return a standardized error envelope with one of these codes: `CONFIG_INVALID`, `ARGS_INVALID`, `TEMPLATE_INVALID`, `TIMEOUT`, `NETWORK`, `HTTP_4XX`, `HTTP_5XX`, `HTTP_UNEXPECTED`, `INVALID_RESPONSE`. `error_mapping` lets an admin override the human-readable message per code.
- `args_schema` is checked against the LLM-supplied `args` before the call is made (a minimal `required` + `properties`. `<name>.type` subset, not full JSON Schema).

Validation

`AgentConfig` (and everything nested under it) is a Pydantic model validated by FastAPI on every `/v1/conversations/process` and `/v1/conversations/stream` request. A body that fails validation returns `422 Unprocessable Entity` with a field-level error message before the orchestration engine runs.

See also: [Agent Task Example](#) · [API Reference](#) · [Agent Configuration Management](#) · [Architecture — Code Components](#)

Agent Task Example

Agent Task Example

A full worked example against the real, current API: a booking assistant that collects restaurant reservation details, using the `AgentConfig` schema described in [Agent Task Schema](#) and the `POST /v1/conversations/process` endpoint described in [API Reference](#).

There is no `backend/tasks/book_appointment.json` file anymore — it was deleted along with the `/pipeline/run` endpoint that loaded it. Agent definitions are sent inline as `AgentConfig` objects; there is nothing to load from disk.

What this agent does

1. Greets the caller with a fixed `greeting` (no LLM call needed for this turn).
2. Collects reservation details — `date`, `time`, `party_size` — via a `data_extraction` tool.
3. Because that tool has `confirmation.required: true`, the conversation pauses and asks the user to confirm before the booking is considered final.
4. The frontend answers back with `confirmation_id` + `decision: "confirm"` on the next turn.

The `AgentConfig`

```
{
  "id": "booking_agent",
  "name": "Booking Assistant",
  "persona": "You help users book restaurant reservations.",
  "greeting": "Hi! I can help you book a table. What day works for you?",
  "stt": { "provider": "deepgram", "language": "multi", "model": "nova-3" },
  "llm": { "provider": "openai", "model": "gpt-4o-mini" },
  "tts": { "provider": "elevenlabs", "voice_id": "21m00Tcm4TlvDq8ikWAM", "model": "eleven_multilingual_v2" },
},
"tools": [
  {
    "id": "book_table",
    "name": "book_table",
    "type": "data_extraction",
    "description": "Collect reservation details",
    "confirmation": {
      "required": true,
      "prompt": "Confirm booking for {party_size} on {date}?"
    },
    "parameters": { "date": null, "time": null, "party_size": null }
  }
],
"workflow_dependencies": [],
"allowed_handoffs": []
}
```

Every field here maps directly to the schema in [Agent Task Schema](#) — `persona` (not `instructions`), `greeting` (not `first_message`), no `variables`, no `engine`, no `vad`. The tool's `confirmation` block is singular and lives on the tool itself, not on the agent.

Turn 1 — request

```
{
  "agent_config": { "...": "the AgentConfig object above" },
  "messages": [
    { "role": "user", "content": "Table for 4 tomorrow at 7pm" }
  ]
}
```

`conversation_id` is omitted — this is the first turn, so the backend auto-creates a conversation and returns its id. Because `greeting` is non-empty, the backend also persists it as the opening assistant message before processing this turn.

Turn 1 — response

```
{
  "conversation_id": "3f2a1c9e-7b41-4d6e-9a10-8c2b5e0d7f61",
  "reply": {
    "role": "assistant",
    "content": "Great, booking a table for 4 tomorrow at 7pm – shall I confirm?"
  },
  "tool_invoked": {
    "id": "book_table",
    "name": "book_table",
    "type": "data_extraction",
    "description": "Collect reservation details",
    "confirmation": { "required": true, "prompt": "Confirm booking for {party_size} on {date}?" },
    "parameters": { "date": "tomorrow", "time": "19:00", "party_size": 4 }
  },
  "confirmation": {
    "confirmation_id": "conf_abc123",
    "action": "book_table",
    "description": "Book table for 4 tomorrow at 7pm"
  },
  "selected_intent": "book_reservation",
  "selected_agent": "booking_agent",
  "intent_confidence": 0.94,
  "status": "confirmation_required",
  "error": "",
  "extracted_slots": { "date": "tomorrow", "time": "19:00", "party_size": 4 },
  "missing_slots": [],
  "locked_intent_name": "book_reservation",
  "next_active_tool_id": "book_table",
  "completed_workflow_steps": [],
  "latency_ms": {
    "stt_processing": null,
    "llm_call": 812.4,
    "llm_api": 790.1,
    "tool_call": 2.1,
    "tts_synthesis": null,
    "total": 815.0,
    "voice_turn": null
  }
}
```

Note `status: "confirmation_required"` — the tool has all slots filled (`missing_slots` is empty) but will not actually run until the user confirms, because `book_table.confirmation.required` is `true`. `latency_ms.stt_processing` and `.tts_synthesis`

are `null` because this was a text-path call through `/v1/conversations/process`, which never touches audio.

Turn 2 — answering the confirmation

The next request carries the `conversation_id` from turn 1, the growing `messages` array, and the confirmation fields defined on `ConversationRequest`:

```
{
  "conversation_id": "3f2a1c9e-7b41-4d6e-9a10-8c2b5e0d7f61",
  "agent_config": { "...": "same AgentConfig as turn 1" },
  "messages": [
    { "role": "user", "content": "Table for 4 tomorrow at 7pm" },
    { "role": "assistant", "content": "Great, booking a table for 4 tomorrow at 7pm – shall I confirm?" },
    { "role": "user", "content": "Yes, that's correct" }
  ],
  "confirmation_id": "conf_abc123",
  "decision": "confirm"
}
```

`decision` is either `"confirm"` or `"reject"` per the `ConversationRequest` schema — see [API Reference](#) for the full field table. The exact reply text and resulting `status` on this second turn depend on the orchestration engine's confirmation-handling logic; only the request shape above is guaranteed by the schema.

Try it yourself

```
curl -X POST http://localhost:8000/v1/conversations/process \
-H "Content-Type: application/json" \
-d '{
  "agent_config": {
    "id": "booking_agent",
    "name": "Booking Assistant",
    "persona": "You help users book restaurant reservations.",
    "greeting": "Hi! I can help you book a table. What day works for you?",
    "stt": {"provider": "deepgram", "language": "multi", "model": "nova-3"},
    "llm": {"provider": "openai", "model": "gpt-4o-mini"},
    "tts": {"provider": "elevenlabs", "voice_id": "21m00Tcm4TlvDq8ikWAM", "model":
"eleven_multilingual_v2"},
    "tools": [
      {
        "id": "book_table",
        "name": "book_table",
        "type": "data_extraction",
        "description": "Collect reservation details",
        "confirmation": {"required": true, "prompt": "Confirm booking for {party_size} on {date}?"},
        "parameters": {"date": null, "time": null, "party_size": null}
      }
    ],
    "workflow_dependencies": [],
    "allowed_handoffs": []
  },
  "messages": [
    {"role": "user", "content": "Table for 4 tomorrow at 7pm"}
  ]
}'
```

Copy `conversation_id` from the response into the next call's `"conversation_id"` field to continue the same conversation.

See also: [Agent Task Schema](#) · [API Reference](#) · [Agent Configuration Management](#)

Agent Configuration Management

API Reference — Agent Configuration

Base URL (local): `http://localhost:8000`

Swagger UI: <http://localhost:8000/docs>

OpenAPI JSON: <http://localhost:8000/openapi.json>

Frontend (dev): <http://localhost:5173/config>

The React app calls `/api/*`. Vite proxies those requests to the backend (see `frontend/vite.config.ts`).

Example: `GET /api/v1/agent-configs` → `GET http://localhost:8000/v1/agent-configs`

Setup (required before testing)

1. Start PostgreSQL:

```
docker compose up postgres -d
```

2. Create env file (if missing):

```
cp backend/.env.example backend/.env
```

Ensure `backend/.env` contains:

```
DATABASE_URL=postgresql://taskorbit:taskorbit@localhost:5435/taskorbit
```

3. Run migrations:

```
cd backend && poetry run alembic upgrade head
```

4. Start the backend (restart after creating `.env`):

```
cd backend && poetry run taskorbit-api
```

5. Start the frontend:

```
cd frontend && npm run dev
```

GET /health

Health check. Returns **200** if the backend is running.

Response:

```
{
  "status": "ok",
  "service": "taskorbit-backend",
}
```

```
"version": "0.1.0"
}
```

Example:

```
curl http://localhost:8000/health
```

Agent configuration overview

Base path: `/v1/agent-configs`

Persist, load, update, and delete agent presets from the Agent Config page.

GitHub issues:

- [#47 Save Agent Configuration](#)
- [#51 Load Agent Configuration](#)
- [#52 Manage Agent Configuration](#)

Storage: PostgreSQL table `agent_configurations`

Config payload: The `config` field is opaque JSON matching the frontend AgentConfig shape (identity, instructions, pipeline, tools, confirmations, variables).

Summary object (list responses)

Used by `GET /v1/agent-configs`. Does not include the `config` blob.

- `id` (string) — database id
- `name` (string) — display name / save label
- `created_at` (string, ISO 8601) — creation time
- `updated_at` (string or null) — last update time

Detail object (single resource)

Used by GET one, POST, and PUT responses. Includes full config.

- `id` (string)
- `name` (string)
- `config` (object) — full agent configuration JSON
- `created_at` (string, ISO 8601)
- `updated_at` (string or null)

GET /v1/agent-configs

List all saved agent configurations (summaries only).

Used by: Load agent dropdown (#51)

Response: `200 OK` — JSON array (newest first)

Example response:

```
[
  {
    "id": "accd7ed5c06e4462a302772f30de3616",
    "name": "John Doe",
    "created_at": "2026-05-19T04:11:09.592904Z",
    "updated_at": null
  }
]
```

```
}  
]
```

Example request:

```
curl http://localhost:8000/v1/agent-configs
```

GET /v1/agent-configs/{config_id}

Load one saved configuration including the full `config` JSON.

Used by: Selecting an entry in the Load agent dropdown (#51)

Path parameter:

- `config_id` (string) — id from list or save response

Response: 200 OK — detail object

Example response:

```
{  
  "id": "accd7ed5c06e4462a302772f30de3616",  
  "name": "John Doe",  
  "config": {  
    "agent_id": "preet-agent",  
    "name": "John Doe",  
    "instructions": "# PERSONA\nYou are John Doe...",  
    "first_message": {  
      "type": "text",  
      "text": "Hi there! I'm John from TechStore..."  
    },  
    "stt": { "provider": "deepgram", "model": "nova-3" },  
    "llm": { "provider": "openai", "model": "gpt-4.1-mini" },  
    "tts": {  
      "provider": "elevenlabs",  
      "model": "eleven_multilingual_v2",  
      "voice_id": "21m00Tcm4TlvDq8ikWAM"  
    },  
    "tools": [],  
    "confirmations": { "enabled": false, "rules": [] },  
    "variables": []  
  },  
  "created_at": "2026-05-19T04:11:09.592904Z",  
  "updated_at": null  
}
```

Error responses:

- 404 Not Found — unknown config_id

Example request:

```
curl http://localhost:8000/v1/agent-configs/accd7ed5c06e4462a302772f30de3616
```

POST /v1/agent-configs

Create and persist a new agent configuration.

Used by:

- Save button when no agent is loaded (#47)
- Save as new when an agent is already loaded (#52) — always creates a new row

Request body:

```
{
  "name": "John Doe",
  "config": {
    "agent_id": "preet-agent",
    "name": "John Doe",
    "instructions": "# PERSONA\n...",
    "first_message": { "type": "text", "text": "..." },
    "stt": { "provider": "deepgram", "model": "nova-3" },
    "llm": { "provider": "openai", "model": "gpt-4.1-mini" },
    "tts": {
      "provider": "elevenlabs",
      "model": "eleven_multilingual_v2",
      "voice_id": "21m00Tcm4TlvDq8ikWAM"
    },
    "tools": [],
    "confirmations": { "enabled": false, "rules": [] },
    "variables": []
  }
}
```

Request fields:

- `name` (string, required) — human-readable label, 1–200 characters
- `config` (object, required) — full frontend agent configuration JSON

Response: 201 Created — detail object with new `id`

Error responses:

- 422 Unprocessable Entity — invalid body (e.g. empty name)
- 500 Internal Server Error — database error (e.g. missing migration)

Example request:

```
curl -X POST http://localhost:8000/v1/agent-configs \
  -H "Content-Type: application/json" \
  -d '{"name":"John Doe","config":{"agent_id":"preet-agent","name":"John Doe","instructions":"# PERSONA\nTest"}}'
```

PUT /v1/agent-configs/{config_id}

Update an existing configuration in place (same id).

Used by: Update button after loading an agent (#52)

Path parameter:

- `config_id` (string) — id of the loaded configuration

Request body:

```
{
  "name": "John Doe Chriss",
  "config": {
    "agent_id": "preet-agent",
    "name": "John Doe Chriss",
    "instructions": "# PERSONA\nUpdated..."
  }
}
```

Request fields:

- `name` (string, optional) — new display name
- `config` (object, optional) — replacement config object

The frontend sends both fields on update.

Response: 200 OK — updated detail object (`updated_at` is set)

Error responses:

- 404 Not Found — unknown `config_id`
- 422 Unprocessable Entity — invalid body

Example request:

```
curl -X PUT http://localhost:8000/v1/agent-configs/accd7ed5c06e4462a302772f30de3616 \
-H "Content-Type: application/json" \
-d '{"name":"John Doe Chriss","config":{"agent_id":"preet-agent","name":"John Doe Chriss","instructions":"# PERSONA\nUpdated"}}'
```

DELETE /v1/agent-configs/{config_id}

Delete a saved configuration.

Used by: Trash icon in the Load agent dropdown (#52). The UI shows a browser confirmation dialog before calling this endpoint.

Path parameter:

- `config_id` (string) — id to delete

Response: 204 No Content (empty body)

Error responses:

- 404 Not Found — unknown `config_id`

Example request:

```
curl -X DELETE http://localhost:8000/v1/agent-configs/accd7ed5c06e4462a302772f30de3616 -i
```

Example — full UI test flow

1. **Save** — POST `/v1/agent-configs` → note the returned `id`
2. **List** — GET `/v1/agent-configs` → see the saved name in the dropdown
3. **Load** — GET `/v1/agent-configs/{id}` → form fields repopulate

- 4. **Update** — PUT /v1/agent-configs/{id} → same id, updated content
- 5. **Save as new** — POST /v1/agent-configs again → second id in the list
- 6. **Delete** — DELETE /v1/agent-configs/{id} → 204 , entry removed from dropdown

Frontend: <http://localhost:5173/config>

Interactive API docs: <http://localhost:8000/docs> (tag: agent-configs)

/v1/user-agents — merged template + customization flow

A second, template-aware persistence path alongside /v1/agent-configs . It backs the agent-config page's "built-in templates + your customizations" list and the copy-on-write "Save"/"Save as new" behavior.

Base path: /v1/user-agents

Method	Path	Behavior
GET	/v1/user-agents	Merged list: the user's customized copies (is_customized: true) plus every active default template not yet customized (is_customized: false). Built-in templates always appear, so the list never shrinks.
GET	/v1/user-agents/templates	Raw default templates only (admin/debug view) — not user-specific.
POST	/v1/user-agents	Always inserts a brand-new row, independent of any template or currently-loaded agent — the correct target for "Save as new".
PUT	/v1/user-agents/{id}	Two paths depending on what {id} is: if it's the primary key of a row the user already owns, that row is updated directly; if it's a template id, the template is cloned into a new row on first save (copy-on-write — {**template.config, **config_updates} — the template itself is never mutated), or the user's existing copy of that template is updated on subsequent saves.
DELETE	/v1/user-agents/{id}	Deletes the user's customized copy only. The agent then reappears as the plain, unmodified template in GET /v1/user-agents.

GET /v1/user-agents response shape (UserAgentResponse):

```
{
  "id": "...",
  "template_id": "sales-agent",
  "name": "My Sales Bot",
  "config": { "...": "opaque JSON, same shape as /v1/agent-configs' config field" },
  "is_default": false,
  "is_customized": true
}
```

user_id is currently a dependency-injected value, not JWT-derived — see the route module's own note (api/routes/user_agents.py).

#203 — Agent-transfer target dropdown

The agent_transfer tool editor in the Agent Config UI (ToolsSection.tsx , AgentTransferEditor) lets an admin pick transfer targets from a dropdown instead of typing raw ids:

- The dropdown is populated from GET /v1/user-agents (merged list — built-in templates and the user's customizations).
- Because a template and the user's customized copy of it are separate rows sharing the same underlying config.agent_id / config.id , entries are **deduplicated for display** by that shared "logical id" —

`list_user_agents_merged` orders the user's own rows before templates, so the customized copy wins the dedup automatically when one exists.

- Selecting an entry stores the row's **stable, resolvable id** (`entry.id` — a hyphenated template slug like `sales-agent` for built-ins, or a UUID hex for custom agents) into `tool.targets` / `parameters.targets`. This is deliberately **not** `config.agent_id` / `config.id`, which are a separate field used only for `workflow_dependencies` / `allowed_handoffs` matching.
- This storage choice was confirmed against the backend's actual resolution logic (#212): both the auto-transfer regex normalization (`orchestration/__init__.py`) and the manual-transfer primary-key lookup (`get_agent_configuration_by_id` / `get_default_agent_template`) resolve transfer targets by this same row-level `id` — so what the dropdown stores is exactly what the backend can look up.
- If the agent list fails to load or is empty, the editor falls back to a free-text input for the target id.

Other live endpoints (TaskOrbit)

These exist in the current backend but are separate from agent configuration:

- `POST /v1/conversations/process` — text conversation turn
- `POST /v1/livekit/token` — LiveKit room token for voice
- `POST /v1/tts/synthesize` — TTS fallback for typed input

For the always up-to-date contract, use Swagger: <http://localhost:8000/docs>

Legacy note (older wiki pages)

Some older documentation referenced:

- `POST /pipeline/run` — not the current main API path
- `POST /livekit/token` without `/v1` prefix — now `POST /v1/livekit/token`

Prefer Swagger (`/docs`) over stale wiki text when in doubt.

Agent Persona Guardrails

Traditional prompt engineering often fails to prevent an agent from answering off-topic questions (e.g. a customer support bot giving medical advice or generating code). TaskOrbit solves this via a two-layer defense mechanism.

Architecture

The guardrails are configured per-agent via the `persona_constraints` block. This block allows an operator to explicitly define the agent's safe zone and strict refusal instructions.

```
{
  "agentDefinition": {
    "metadata": {
      "persona_constraints": {
        "scope": "TechStore customer service: account setup, order tracking, returns, and product questions.",
        "out_of_scope": [
          "medical advice",
          "therapy or emotional counseling",
          "legal advice",
          "financial advice"
        ],
        "refusal_template": "I'm here to help with TechStore questions – for that I'd recommend reaching out to a qualified professional. Is there anything TechStore-related I can help with?"
      }
    }
  }
}
```

These three fields ride along inside `AgentConfig.persona_constraints`. Backend Pydantic model: `taskorbit.types.PersonaConstraints`.

1. **Frontend "Belt-and-Suspenders"**: The configuration UI injects a `CRITICAL RULES` block at the very top of the agent's instructions. This explicitly locks the role and forbids advice drift at the instruction level.
2. **Backend Prompt Hardening**: The `ConversationOrchestrator` intercepts the prompt before calling the LLM and appends the constraints using highly imperative, authoritative language (e.g. `CORE CONSTRAINT - Authorized Scope`). This forces the LLM to prioritize the refusal rules over conversational drift.

Example Scenarios

The following scenarios document the expected behavior of the agent when persona guardrails are active. These were verified against the "John Doe" TechStore demo configuration.

Scenario 1: Happy Path (In Scope)

- **User**: "Can you tell me the status of my order #12345?"
- **Agent Behavior**: Processes the request normally.
- **Agent Response**: "I'd be happy to check that for you. Let me look up order #12345..."

Scenario 2: Edge Case — Medical Advice

- **User**: "I have a really bad headache and my throat is sore. What should I take?"
- **Agent Behavior**: Triggers the out-of-scope guardrail.
- **Agent Response**: "I'm here to help with TechStore questions — for that I'd recommend reaching out to a qualified professional. Is there anything TechStore-related I can help with?"

Scenario 3: Edge Case — Emotional Counseling / Therapy

- **User**: "I'm just feeling really sad today and I don't know what to do."

- **Agent Behavior:** Triggers the out-of-scope guardrail.
- **Agent Response:** "I'm here to help with TechStore questions — for that I'd recommend reaching out to a qualified professional. Is there anything TechStore-related I can help with?"

Scenario 4: Edge Case — Legal Advice

- **User:** "My landlord is trying to evict me without notice. Is that legal?"
- **Agent Behavior:** Triggers the out-of-scope guardrail.
- **Agent Response:** "I'm here to help with TechStore questions — for that I'd recommend reaching out to a qualified professional. Is there anything TechStore-related I can help with?"

Scenario 5: Edge Case — Financial Advice

- **User:** "Which stocks should I invest in right now? Apple or Google?"
- **Agent Behavior:** Triggers the out-of-scope guardrail.
- **Agent Response:** "I'm here to help with TechStore questions — for that I'd recommend reaching out to a qualified professional. Is there anything TechStore-related I can help with?"

Scenario 6: Mixed Input (Emotional + In Scope)

- **User:** "I'm so frustrated and angry! My package was supposed to be here yesterday and it's ruined my whole week!"
- **Agent Behavior:** Recognizes the frustration but stays in scope because it relates to an order.
- **Agent Response:** "I completely understand your frustration, and I apologize for the delay. Let's look into your order right away to see what happened."

Implementation Mapping

Concept	Location	Identifier
Canonical JSON schema	schemas/agent-task.schema.json	agentDefinition.metadata.persona_constraints + \$defs/personaConstraints
Python runtime model	backend/src/taskorbit/types.py	PersonaConstraints + AgentConfig.persona_constraints
Orchestrator injection	backend/src/taskorbit/orchestration/__init__.py	ConversationOrchestrator._build_system_prompt()
Prompt helper	backend/src/taskorbit/integrations/llm/prompts.py	with_persona_guardrails()
FE seed (demo)	frontend/src/lib/mockAgents.ts	JOHN_DOE_AGENT.persona_constraints

End Call Tool

The `end_call` tool lets the agent gracefully hang up when the user explicitly requests it. This page covers how to enable it, which phrases trigger it, and how the detection works.

Enabling the tool

In the agent configuration editor, open the **Tools** section and add an **End call** tool. The tool name must be `end_call`. The description field is optional — a default is used if left blank.

Once saved, every new voice or text session started for that agent will have end-call detection active.

How detection works

Detection runs **before** intent routing on every user turn. The backend checks whether the user's message contains any of the trigger phrases below (case-insensitive, substring match). If it does, the LLM generates a short farewell and the session ends immediately — no slot collection, no agent transfer.

Negation guard: if a trigger phrase is immediately preceded by a negation word (`don't` , `do not` , `won't` , `cannot` , `not` , `never` , etc.), the match is skipped. For example, *"please don't end the call"* does **not** trigger end-call.

Trigger phrases

Unambiguous farewells

Phrase
goodbye
good bye
bye bye

Explicit end-call requests

Phrase
end the call
end this call
end call
end the conversation
end this conversation
close the call
terminate the call
hang up
hangup
i want to hang up
i want to end the call
i want to end this call

please end the call
please hang up

Clearly finished

Phrase
that's all i needed
that is all i needed
that's everything i needed
that is everything i needed
no more questions
i have no more questions
i'm done for now
im done for now
we're done here
were done here
i'm ready to end
im ready to end

Wrap-up with call reference

Phrase
wrap up the call
let's end the call
lets end the call

Done for the day / session

Phrase
done for the day
done for today
i'm done for the day
im done for the day
i am done for the day
am done for the day
i think i'm done
i think im done
i think am done

i think i am done
i think that's all
i think thats all
i think that's everything
i think thats everything
i think we're done
i think were done
i guess that's all
i guess thats all
that's it for today
thats it for today

Negation examples

User message	Triggers end-call?
"goodbye"	Yes
"can you please end the call?"	Yes (contains <i>end the call</i>)
"okay, goodbye then"	Yes (contains <i>goodbye</i>)
"please don't end the call"	No (negation guard)
"don't hang up"	No (negation guard)
"do not end the conversation"	No (negation guard)

Farewell message

When end-call fires, the LLM generates a one-sentence farewell in the agent's persona. If the LLM is unavailable or times out, the session still ends with a hardcoded fallback: *"Goodbye! Take care."*

Adding new phrases

Trigger phrases are defined in `_END_CALL_SIGNALS` inside `backend/src/taskorbit/orchestration/__init__.py`. Add the lowercase phrase to the frozenset. Negation words are defined in the adjacent `_NEGATION_PREFIXES` frozenset.

New phrases should be covered by the parametrized tests in `backend/tests/test_orchestration.py` (`test_user_requested_end_call_matches_signals`).

Generic External Tool Adapter

Generic External Tool Adapter

Overview

This document describes the **Generic External Tool Adapter** implemented for TaskOrbit as part of ticket #66. The adapter lets system admins give an agent the ability to call an external HTTP API (REST endpoint, outbound webhook, CRM lookup, etc.) by writing a JSON configuration block instead of new Python code.

Before #66, every external API the agent could call needed its own `BaseTool` subclass. Adding a new API meant a code change, a PR, a review, and a deploy. The adapter collapses that loop into a single config edit on the agent.

Problem Statement

The platform supported three concrete tool types (`data_extraction` , `agent_transfer` , `end_call`), each implemented as its own Python class. Any new integration the industry partner wanted (weather lookup, CRM record fetch, Slack/Zapier outbound webhook, internal microservice call, etc.) required:

- A new file under `backend/src/taskorbit/tools/`
- A new entry in `ToolType`
- A new row in the dispatch table
- A new FE form variant
- A code review, a deploy, and a release

The IP partner explicitly asked for a **generic** adapter (per the 2026-06-08 IP meeting) rather than per-API connectors, so the design is deliberately not opinionated towards any single vendor.

Architecture

Pipeline per Tool Call

```
LLM decides to call an external_api tool
|
▼
1. parse_config(tool.parameters)
|   ├── malformed → ToolResult{ error_code: CONFIG_INVALID }
|   └── ok
|
▼
2. validate args against args_schema
|   ├── missing / wrong type → ToolResult{ error_code: ARGS_INVALID }
|   └── ok
|
▼
3. substitute templates {{env.X}} + {{args.Y}}
|   ├── unwhitelisted env or missing arg → ToolResult{ error_code: TEMPLATE_INVALID }
|   └── ok
|
▼
4. httpx.AsyncClient.request(method, url, headers, query, body, timeout)
|   ├── TimeoutException → ToolResult{ error_code: TIMEOUT }
|   ├── RequestError (DNS/SSL/conn) → ToolResult{ error_code: NETWORK }
|   └── ok
|
▼
5. parse response body as JSON
|   ├── non-JSON + extract configured → ToolResult{ error_code: INVALID_RESPONSE }
|   └── ok
|
▼
```

```

6. status check against response.success_when.status_in
|   ├── 400-499 → ToolResult{ error_code: HTTP_4XX }
|   ├── 500-599 → ToolResult{ error_code: HTTP_5XX }
|   ├── other not-success → ToolResult{ error_code: HTTP_UNEXPECTED }
|   └── success
▼
7. apply response.extract (dot-path walk) → standardised data shape
▼
ToolResult{ success: true, data: {status, data, raw, headers} }

```

Key Components

Component	File	Role
GenericApiTool	backend/src/taskorbit/tools/generic_api.py	The single BaseTool subclass interpreting the config at runtime
parse_config	same file	Validates the raw config dict and returns a frozen GenericApiConfig
substitute_string / substitute_tree	same file	{{env.X}} and {{args.Y}} template substitution with env-var whitelist
extract_path / extract_response	same file	Dot-path walk into the upstream JSON body
build_error	same file	Builds the standardised error envelope {error_code, error_message, error_detail, status}
ToolType.EXTERNAL_API	backend/src/taskorbit/types.py	Enum value used by the dispatch table
_dispatch_tool	backend/src/taskorbit/orchestration/__init__.py	Routes external_api tools to GenericApiTool
ExternalApiTool	frontend/src/types/agentConfig.ts	FE type definition for the new variant
ExternalApiEditor	frontend/src/components/agent-config/ToolsSection.tsx	FE form for authoring external API tools
externalApiTool (schema)	schemas/agent-task.schema.json	JSON Schema validating saved agent configs

How to Add an External Tool

You edit the tool's `parameters` block directly in the agent config — either through the **Tools** section of the Agent Configuration page (recommended) or by editing the saved JSON.

Worked Example: Weather Lookup

```

{
  "type": "external_api",
  "name": "lookup_weather",
  "description": "Use this tool to look up current weather for a city.",
  "parameters": {
    "request": {
      "method": "GET",
      "url": "https://api.weatherapi.com/v1/current.json",
      "headers": {
        "X-API-Key": "{{env.WEATHER_API_KEY}}"
      }
    }
  }
}

```

```

    },
    "query": {
      "q": "{{args.city}}"
    },
    "timeout_seconds": 5
  },
  "response": {
    "extract": {
      "temperature_c": "current.temp_c",
      "condition": "current.condition.text"
    },
    "success_when": {
      "status_in": [200]
    }
  },
  "auth": {
    "allowed_env": ["WEATHER_API_KEY"]
  },
  "error_mapping": {
    "TIMEOUT": "The weather service is slow right now. Try again in a moment.",
    "HTTP_4XX": "I could not find weather data for that location."
  },
  "args_schema": {
    "type": "object",
    "required": ["city"],
    "properties": {
      "city": {
        "type": "string",
        "description": "City name in English."
      }
    }
  }
}

```

When the LLM decides to call this tool with `{"city": "Berlin"}`, the adapter substitutes templates, sends `GET https://api.weatherapi.com/v1/current.json?q=Berlin` with the API key header, and returns:

```

{
  "status": 200,
  "data": {
    "temperature_c": 18.3,
    "condition": "Cloudy"
  },
  "raw": { /* full upstream payload preserved for debugging */ },
  "headers": { /* response headers */ }
}

```

Config Reference

request (required)

Field	Type	Required	Notes
method	string	yes	GET, POST, PUT, PATCH, DELETE

url	string	yes	Absolute URL; supports {{env.X}} and {{args.Y}}
headers	object	no	String values, substitution supported
query	object	no	String values, substitution supported
body	any	no	JSON body (object / array / primitive); substitution walks nested strings
timeout_seconds	number	no	Default 10, must be > 0

response (optional)

Field	Type	Notes
extract	object	Maps stable output names to dot-paths in the response body
success_when.status_in	array	List of integer statuses considered success. Default [200..299]

auth (optional but security-relevant)

Field	Type	Notes
allowed_env	array	Whitelist of env-var names that may appear in {{env.NAME}}. Required when using env

error_mapping (optional)

Map of `ERROR_CODE` -> `human-friendly message` . Overrides the default message for that code (see error code list below).

args_schema (optional but strongly recommended)

Minimal JSON-Schema-style description of the args the LLM must supply. Supports `required` and `properties.<name>.type` (`string` , `integer` , `number` , `boolean` , `object` , `array`).

Template Substitution

Two namespaces are recognised:

- `{{args.NAME}}` — runtime arguments supplied by the LLM. Nested keys via dot syntax: `{{args.user.id}}` .
- `{{env.NAME}}` — process environment variables. **NAME must be listed in `auth.allowed_env`** or substitution fails with `TEMPLATE_INVALID` . This whitelist prevents a careless or malicious config from exfiltrating secrets like `AWS_SECRET_ACCESS_KEY` .

Substitution applies recursively to strings inside `url` , `headers` , `query` , and `body` . Missing args or unauthorised env vars raise immediately rather than silently sending an incomplete request.

Response Extraction

The `response.extract` map pulls fields out of the upstream JSON and exposes them under stable names. Path syntax is **dot-separated**:

```
"extract": {
  "temperature": "current.temp_c",
  "humidity": "current.humidity"
}
```

walks `body["current"]["temp_c"]` and `body["current"]["humidity"]` .

Path syntax	Supported?	Behaviour
-------------	------------	-----------

<code>data.nested.field</code>	yes	Dot-walk into nested dicts
<code>\$.data.nested</code>	yes	Leading <code>\$. </code> accepted and stripped
<code>\$</code> alone	yes	Returns the whole body
<code>items[0].name</code>	no (v1)	Array indexing not yet supported
<code>\$.items[*].name</code>	no (v1)	Wildcards not yet supported

When `extract` is configured, the standardised view lives under `data` and the original upstream payload lives under `raw` so operators can still inspect what came back.

Missing segments resolve to `null` rather than raising. A single missing field will not blow up an otherwise-successful response.

Error Codes

Every failure surfaces under a standardised envelope:

```
{
  "error_code": "HTTP_4XX",
  "error_message": "The CRM could not find that record.",
  "error_detail": "upstream returned HTTP 404",
  "status": 404
}
```

Code	When it fires
CONFIG_INVALID	Config does not match the schema (unsupported method, missing URL, etc.)
ARGS_INVALID	LLM args violate <code>args_schema</code> (missing required key, wrong type)
TEMPLATE_INVALID	<code>{{env.X}}</code> reference not in <code>auth.allowed_env</code> , or <code>{{args.Y}}</code> key not supplied
TIMEOUT	HTTP request exceeded <code>request.timeout_seconds</code>
NETWORK	Connection refused, DNS failure, SSL error — anything below the response layer
HTTP_4XX	Response status 400-499 and not in <code>success_when.status_in</code>
HTTP_5XX	Response status 500-599 and not in <code>success_when.status_in</code>
HTTP_UNEXPECTED	Status is neither success nor 4xx/5xx (e.g. 3xx, or 201 when only 200 was allowed)
INVALID_RESPONSE	<code>extract</code> is configured but the response body is not parseable JSON

`error_mapping` overrides the human-readable `error_message` per code. `error_detail` always carries the technical reason for logs and debugging, regardless of mapping.

Security Model

The adapter is config-driven, which means a tool config is effectively code. Two safety mechanisms keep the blast radius contained:

1. **Env-var whitelist** (`auth.allowed_env`). A `{{env.X}}` template reference fails with `TEMPLATE_INVALID` if `X` is not on the list, so a typo or a malicious config cannot leak unrelated secrets.
2. **Load failures on missing data**. Missing args or unset env vars raise before the HTTP request fires, so the agent does not accidentally call an external service with an `Authorization: Bearer` header and an empty key.

API keys themselves still live in the process environment, never in the config. The config only references their names.

Webhooks

Confirmed scope per the 2026-06-08 IP meeting + Discord confirmation:

Direction	Supported?	Notes
Outbound (we call a webhook URL, e.g. Slack incoming, Zapier catch hook)	Yes	Just POST + URL + body template, no new architecture
Inbound (an external service POSTs events to us)	No	Needs its own ticket (public endpoint, signature verification, event routing into active sessions)

Testing

Running Tests

```
cd backend

# Unit tests
poetry run pytest tests/test_generic_api_tool.py -v

# Schema tests
# Run from the host with the full repo checked out, or via CI. Do not run
# inside `docker compose exec backend` because the `schemas/` directory lives
# at the project root and is not mounted into the backend container.
poetry run pytest tests/test_agent_task_schema.py -v

# Integration tests against real httpbin.org (requires internet)
poetry run pytest -m integration
```

Test Coverage

File	What It Covers
tests/test_generic_api_tool.py	parse_config, template substitution, env-var whitelist, args validation, httpx execution (mocked), response extraction, full error taxonomy
tests/test_agent_task_schema.py	JSON Schema validates the new external_api variant, rejects bad methods, enforces additionalProperties=false
tests/test_generic_api_integration.py	Real HTTP round-trip against httpbin.org/anything and httpbin.org/status/404

Integration Testing Manually

1. Start the backend: `docker compose up backend`
2. Open the Agent Configuration page (`http://localhost:5173/config`)
3. Pick "External API" from the tool type dropdown, click Add tool
4. Paste the weather example above (or any other config)
5. Save the agent, switch to chat, send a message that should trigger the tool
6. Check the worker log for `generic_api_request` and `generic_api_response` events

Known Limitations / Future Work

These were intentionally not built in #66 so the first version stays focused. Open a new ticket if a real config needs any of them.

Item	Notes
------	-------

OAuth2 / refresh tokens	Auth is API-key / static header only
File upload / multipart bodies	JSON body only
Streaming responses	Single request → single response only
Retry-with-backoff	No transient-failure retry; tool returns the error directly
Per-tool rate limiting	None
Full JSONPath in extraction	Dot-path + \$. prefix only; no [*], no filters
Inbound webhooks	Adapter handles outbound for free; inbound needs its own ticket (public endpoint, signature verification, event routing into active sessions)
Mid-call agent / tool hot reload on voice	Voice worker still loads the agent config once at room start; new tools added mid-call are not picked up until reconnect

See Also

- `Documentation/external-api-tools.md` in the main repo — same content lives in-tree for discoverability from the codebase
 - `[[Agent Routing Foundation]]` — how the orchestrator decides which tool to dispatch
 - `[[Agent Task Schema]]` — the JSON contract this tool variant extends
 - `[[Agent Configuration Management]]` — how saved agent configs round-trip through the FE
-

API Reference

API Reference

Base URL (local): `http://localhost:8000` Auto-generated docs (Swagger UI): `http://localhost:8000/docs` OpenAPI JSON: `http://localhost:8000/openapi.json`

All routes below are mounted with a `/v1` prefix except `/health`. There is no `/pipeline/run` and no `/sessions` router in the current backend — both were removed; conversations are addressed by `conversation_id`, not a `session_id`.

Endpoint summary

Method	Path	Purpose
GET	<code>/health</code>	Liveness probe
POST	<code>/v1/conversations/process</code>	Main endpoint. One conversation turn, full JSON response
POST	<code>/v1/conversations/stream</code>	One conversation turn, Server-Sent Events token stream
POST	<code>/v1/conversations</code>	Create a bare conversation, returns a new <code>conversation_id</code>
GET	<code>/v1/conversations</code>	List all conversations
GET	<code>/v1/conversations/{id}/history</code>	Full history: messages, tool executions, slot extractions
GET	<code>/v1/conversations/{id}/messages</code>	Messages only
DELETE	<code>/v1/conversations/{id}</code>	Permanently delete a conversation
GET	<code>/v1/agent-configs</code>	List saved agent configuration summaries
POST	<code>/v1/agent-configs</code>	Save a new agent configuration
GET	<code>/v1/agent-configs/{id}</code>	Load one configuration, full detail
PUT	<code>/v1/agent-configs/{id}</code>	Update a configuration in place
DELETE	<code>/v1/agent-configs/{id}</code>	Delete a configuration
GET	<code>/v1/user-agents</code>	Merged list — user's customized copies + untouched templates
GET	<code>/v1/user-agents/templates</code>	Raw default templates only
POST	<code>/v1/user-agents</code>	"Save as new" — always inserts a fresh agent
PUT	<code>/v1/user-agents/{id}</code>	Update-or-clone (copy-on-write against a template)
DELETE	<code>/v1/user-agents/{id}</code>	Delete the user's copy; agent reverts to template view
POST	<code>/v1/livekit/token</code>	Mint a signed LiveKit room access token
POST	<code>/v1/tts/synthesize</code>	Text → MP3 audio (ElevenLabs)
POST	<code>/v1/tts/synthesize-with-timestamps</code>	Text → MP3 + word-level timestamps

For the full request/response schema of agent configuration persistence (`/v1/agent-configs`), see [Agent Configuration Management](#). For the `AgentConfig` object shape used in conversation requests, see [Agent Task Schema](#).

GET `/health`

Liveness probe used by Docker, the frontend, and tests.

Response

```
{ "status": "ok", "service": "taskorbit-backend", "version": "0.1.0" }
```

POST /v1/conversations/process — main endpoint

Process one turn of a conversation through the orchestration engine and persist the messages, tool execution, and any extracted slots. If `conversation_id` is omitted or unknown, the backend auto-creates a conversation and returns the new id — the frontend never has to call `POST /v1/conversations` first. If the agent config's `greeting` is set, it is persisted as the opening assistant message on that first call.

Request body — ConversationRequest

Field	Type	Required	Description
<code>conversation_id</code>	string	—	Omit on the first turn; the server assigns and returns one
<code>agent_config</code>	AgentConfig	✓	The full agent definition for this conversation
<code>messages</code>	array of {role, content, timestamp?}	✓	Conversation history including the newest user message
<code>current_intent_name</code>	string	—	Currently locked intent, if any
<code>selected_agent</code>	string	—	Currently routed agent, if any
<code>active_tool_id</code>	string	—	Tool awaiting completion, if any
<code>confirmation_id</code>	string	—	Echo of a pending <code>confirmation.confirmation_id</code>
<code>decision</code>	"confirm" "reject"	—	User's answer to a pending confirmation
<code>completed_workflow_steps</code>	array of string	—	Workflow blocks already completed
<code>dependency_configs</code>	object (agent id → AgentConfig)	—	Full configs for workflow dependency resolution
<code>manual_transfer</code>	{target_agent_id?, target_agent_name?}	—	UI-initiated handoff that bypasses intent detection

Response body — ConversationResponse

Field	Type	Description
<code>conversation_id</code>	string	Id for this conversation — pass back on every subsequent turn
<code>reply</code>	{role, content, timestamp?}	The agent's message for this turn
<code>tool_invoked</code>	object or null	The <code>ToolDefinition</code> that fired this turn, if any
<code>confirmation</code>	{confirmation_id, action, description} Or null	Present when a tool requires user confirmation before running
<code>selected_intent</code>	string	Detected intent name
<code>selected_agent</code>	string	Agent that handled this turn
<code>intent_confidence</code>	float	Confidence score for <code>selected_intent</code>

status	enum	See status values below
error	string	Technical error detail (empty on success)
extracted_slots	object	Slot values extracted so far
missing_slots	array of string	Required slots not yet filled
locked_intent_name	string or null	Intent locked for the remainder of the conversation
next_active_tool_id	string or null	Tool expected to run next
completed_workflow_steps	array of string	Workflow blocks completed so far
latency_ms	object or null	Per-stage timings: stt_processing, llm_call, llm_api, tool_call, tts_synthesis, total, voice_turn (each nullable — unmeasured stages on the text path are null)

status values: success, clarification, confirmation_required, workflow_confirmation_required, handoff_blocked, rejected, ended, error

Example — see [Agent Task Example](#) for a full worked request/response using a real booking_agent config.

Error responses

Status	When
422 Unprocessable Entity	Request body fails ConversationRequest validation
500 Internal Server Error	Conversation could not be created, or a message failed to persist
501 Not Implemented	Orchestration engine raised NotImplementedError for this path

POST /v1/conversations/stream

Identical inputs to /process, but streams the reply token-by-token over Server-Sent Events instead of waiting for the full response. The user message is persisted immediately so it survives client disconnects; the assistant reply is persisted once the stream completes.

Event schema

```
data: {"type": "chunk", "text": "<token>"}
data: {"type": "done", "intent": "...", "status": "...", "selected_agent": "...",
      "slots": {...}, "missing_slots": [...], "conversation_id": "...",
      "locked_intent_name": "...", "next_active_tool_id": "...",
      "tool_invoked": {...} | null, "completed_workflow_steps": [...],
      "confirmation": {...} | null, "reply": "..."}
data: {"type": "error", "message": "<technical>", "reply": "<polite user-facing text>"}
```

On short-circuit paths (clarification, a confirmation prompt, a handoff block, or a manual transfer) no chunks are streamed — the full reply text is emitted as a single chunk event immediately before done.

POST /v1/conversations

Create a bare conversation ahead of the first turn. Most callers don't need this — /process and /stream auto-create one when conversation_id is omitted.

Response (201 Created)

```
{ "conversation_id": "3f2a1c9e-...", "started_at": "2026-07-12T10:00:00+00:00" }
```

GET /v1/conversations

List all conversations, most recent first.

```
{ "conversations": [ { "id": "...", "agent_name": "...", "started_at": "...", "ended_at": null } ] }
```

GET /v1/conversations/{id}/history

Full history for one conversation: messages, tool executions, and slot extractions. `404` if the conversation doesn't exist.

GET /v1/conversations/{id}/messages

Messages only, in order.

DELETE /v1/conversations/{id}

Permanently deletes the conversation and all associated messages, tool executions, and slot extractions. Returns `204 No Content` ; `404` if the conversation doesn't exist.

POST /v1/livekit/token

Mint a signed LiveKit room-access JWT for the voice UI. Requires `LIVEKIT_URL` , `LIVEKIT_API_KEY` , and `LIVEKIT_API_SECRET` to be configured server-side (`503` if not). Accepts `identity` , `room` , optional `metadata` (capped at 4096 bytes serialized), and optional `agent_dispatch_name` to target a specific LiveKit Cloud agent.

POST /v1/tts/synthesize

Returns raw MP3 audio (`audio/mpeg`) for `{text, voice_id?, model_id?}` via ElevenLabs. `503` if no ElevenLabs API key is configured; `502` if ElevenLabs itself errors.

POST /v1/tts/synthesize-with-timestamps

Same as above but returns `{audio_base64, word_timestamps: [{word, start_time, end_time}]}` so a UI can highlight words as they're spoken.

Agent configuration endpoints

See [Agent Configuration Management](#) for the full `/v1/agent-configs` contract (request/response bodies, error codes, worked curl examples).

`/v1/user-agents` is a separate, template-aware router:

- `GET /v1/user-agents` returns every agent visible to a user: their customized copies (`is_customized: true`) plus every active default template the user hasn't touched (`is_customized: false`). The list never shrinks as templates are added.
- `GET /v1/user-agents/templates` returns the raw default templates only (admin/debug view).
- `POST /v1/user-agents` always inserts a brand-new row ("Save as new") — it never modifies an existing agent.
- `PUT /v1/user-agents/{id}` does one of two things depending on what `{id}` is:
 - if it's the primary key of a row the user already owns, that row is updated directly;
 - if it's a template id, the template is cloned into a new row for that user on first save (copy-on-write; the template itself is never mutated), or the user's existing copy of that template is updated on subsequent saves.
- `DELETE /v1/user-agents/{id}` removes the user's customized copy only; the agent then reappears as the plain template in `GET /v1/user-agents` .

Both `agent-configs` and `user-agents` store the `config` field as an **opaque JSON blob** (`dict[str, Any]`) — it is not validated against the strict `AgentConfig` schema described in [Agent Task Schema](#). That strict schema is the orchestrator's wire type, enforced only on `ConversationRequest.agent_config` in `/v1/conversations/process` and `/v1/conversations/stream` .

See also: [Getting Started](#) · [Agent Task Schema](#) · [Agent Task Example](#) · [Agent Configuration Management](#) · [System Architecture](#)